Morse Code Converter

Deep Dive

Explainer document for the owner

Contents

1 What this project is

This is a small Python project with two parts:

- `morse_converter.py` — pure conversion logic: a lookup table mapping every supported character to its International Morse Code pattern, and two functions that convert text to Morse and Morse back to text. It has no user-interface code at all.
- `gui.py` — a desktop window (built with **Tkinter**, the graphical toolkit that ships with standard Python) that wraps those two functions in a live, two-way interface: you type in one box and the conversion appears in another box as you type, plus a small animation that visually "plays back" the dots and dashes of the current Morse code.

Glossary term — Morse code: a way of encoding text as sequences of short and long signals ("dots" and "dashes"), historically tapped out on a telegraph key or flashed as light. Each letter, digit, or punctuation mark has its own fixed sequence, defined by international convention (this project uses the standard ITU alphabet).

Glossary term — dictionary (Python): a data structure that stores key-value pairs and looks up a value instantly given its key, similar to a paper dictionary mapping a word to its definition. This project uses one dictionary to go from character to Morse pattern, and a second, automatically built by reversing the first, to go the other way.

2 The conversion logic (`morse_converter.py`)

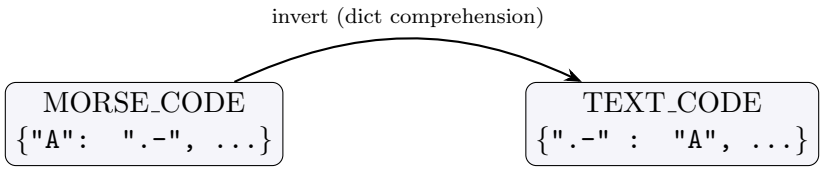
2.1 The lookup tables

The heart of the file is a single dictionary, `MORSE_CODE`, with one entry per supported symbol: the 26 uppercase letters, the 10 digits, the space character (mapped to `"/"`, explained below), and a set of punctuation marks (`! ? . , ' " / ( ) & : ; = + - _ $ @` and two accented question/exclamation marks, ¿ and ¡). Every value is a string made only of the characters `.` (dot) and `-` (dash).

A second dictionary, `TEXT_CODE`, is built once, at import time, by inverting `MORSE_CODE`:

```
TEXT_CODE = {value: key for key, value in MORSE_CODE.items()}
```

This is a Python *dict comprehension*: a compact way of writing a loop that builds a new dictionary. Reading it as an ordinary loop, it means: "for every (character, pattern) pair already in `MORSE_CODE`, add a new entry to `TEXT_CODE` keyed by the pattern, pointing back at the character." Because every Morse pattern in the table is unique, this inversion loses no information: given any valid pattern, `TEXT_CODE` recovers exactly one character.

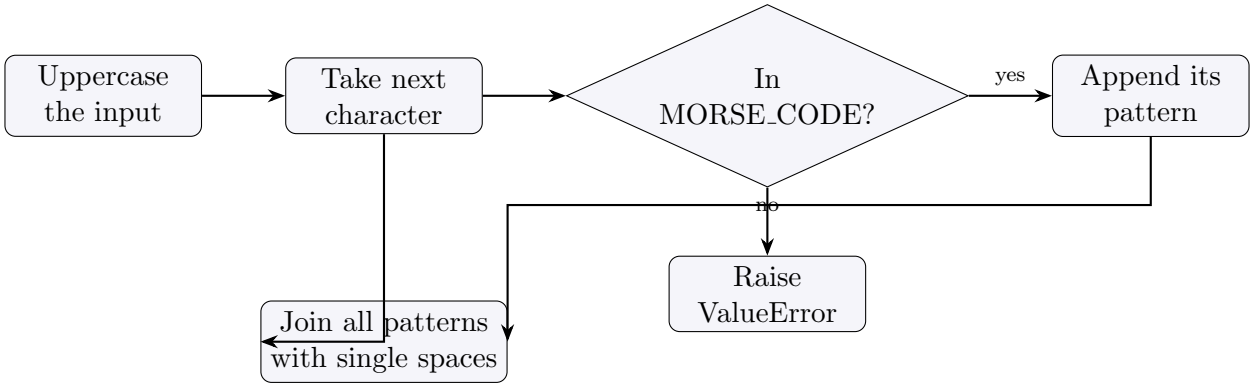


2.2 `text_to_morse(text)`: text → Morse

```
def text_to_morse(text: str) -> str:
    codes = []
    for char in text.upper():
        if char not in MORSE_CODE:
            raise ValueError(f"No Morse mapping for character: {char!r}")
        codes.append(MORSE_CODE[char])
    return " ".join(codes)
```

Step by step:

1. **Normalize case.** `text.upper()` converts the whole input to uppercase first, so `"sos"` and `"SOS"` produce the identical result — the table only has uppercase letter keys.
2. **Look up each character.** The function walks the string one character at a time. If a character has no entry in `MORSE_CODE` (an emoji, an unsupported accented letter, etc.), it deliberately raises a `ValueError` — a built-in Python exception used to signal "this input is invalid" — naming the exact offending character, rather than letting the lookup fail with a raw, less helpful `KeyError` (Python's default error when a dictionary key is missing).
3. **Join with spaces.** Each character's Morse pattern is collected into a list, then joined with single spaces. The space character itself maps to `"/"`, so a literal space in the input becomes the three characters `" / "` in the output once joined (a space before the `/`, the `/` itself, and the space after it from the join). This is the conventional Morse word separator.



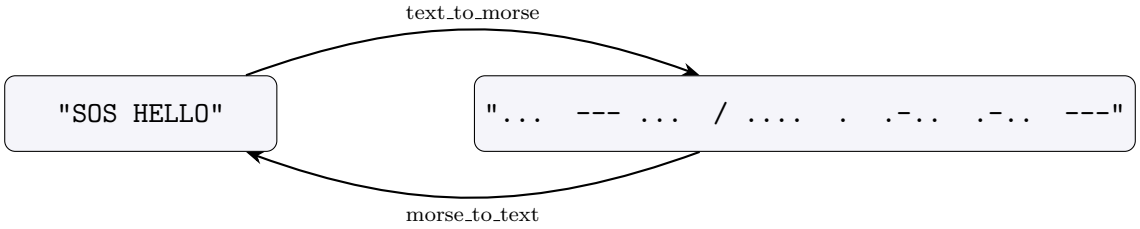
2.3 `morse_to_text(morse)`: Morse → text

```
def morse_to_text(morse: str) -> str:
    words = morse.strip().split(" / ")
    decoded_words = []
    for word in words:
        letters = []
        for symbol in word.split():
            if symbol not in TEXT_CODE:
                raise ValueError(f"No text mapping for Morse symbol: {symbol!r}")
            letters.append(TEXT_CODE[symbol])
        decoded_words.append("".join(letters))
    return " ".join(decoded_words)
```

This function reverses `text_to_morse` in two nested passes:

1. **Split into words.** `morse.strip()` removes leading and trailing whitespace, then `.split(" / ")` cuts the string on the exact three-character sequence `" / "` (space, slash, space). This exact-string split matters: it is precisely what `text_to_morse` produces at a word boundary (see §2.2), so splitting on it, rather than on a bare `"/"`, avoids leaving stray leading/trailing spaces attached to the words on either side.
2. **Split each word into letters.** Within a word, `.split()` with no argument splits on any run of whitespace, giving one Morse pattern per letter (e.g. `"....  .  .-..  .-..  ---"` becomes five patterns).
3. **Look up and rebuild.** Each pattern is looked up in `TEXT_CODE`; an unmapped pattern raises `ValueError` naming the exact symbol, mirroring the error handling in `text_to_morse`. The recovered letters are concatenated with no separator (`"".join(letters)`) to rebuild the word, and the recovered words are joined back together with ordinary single spaces.

2.4 Round trip



Because `text_to_morse` always uppercases its input, `morse_to_text(text_to_morse(s)) == s.upper()`, not necessarily `s` itself if `s` contained lowercase letters.

2.5 Running the file directly

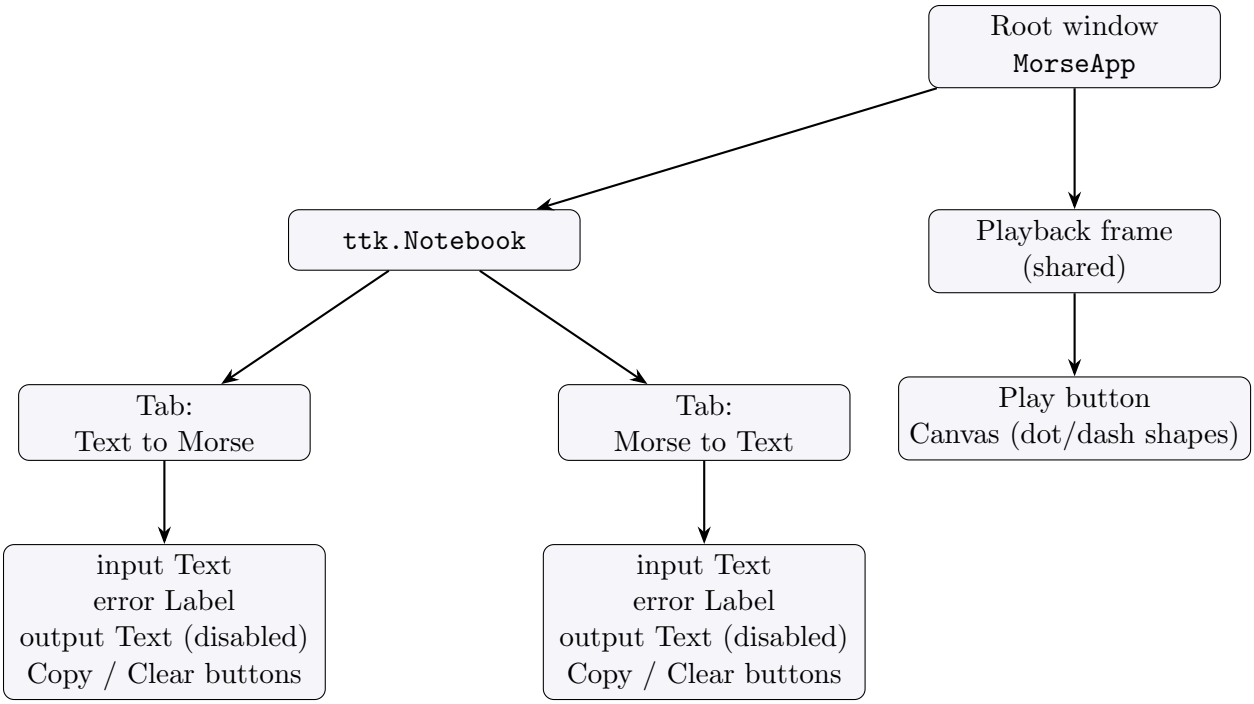
The file's `if __name__ == "__main__":` block — Python's convention for "only run this when the file is executed directly, not when it is imported by another file" — still contains the original one-line command-line prompt: it asks for a message with `input()` and prints the Morse translation. This still works exactly as before; `gui.py` does not replace it, it imports the two functions and builds a separate interface around them.

3 The GUI (`gui.py`)

Glossary term — Tkinter: Python's standard, built-in library for building desktop windows with buttons, text boxes, and other controls ("widgets"). It ships with the standard CPython installer on Windows and macOS; on Linux it is sometimes a separate OS package (`python3-tk`).

Glossary term — `ttk`: a themed extension of Tkinter's basic widgets (buttons, tabs, frames) that supports more modern-looking styling; `gui.py` uses it for the tab bar, buttons, and frames, while using plain Tkinter widgets for the multi-line text boxes and the canvas.

3.1 Widget tree



Both tabs are built by the same helper function, `_build_conversion_tab`, called twice with different labels and a different conversion function (`_safe_text_to_morse` or `_safe_morse_to_text`, which are thin wrappers that just call `text_to_morse`/`morse_to_text`). This is why the two tabs look and behave identically except for which direction they convert: they share one code path instead of being duplicated.

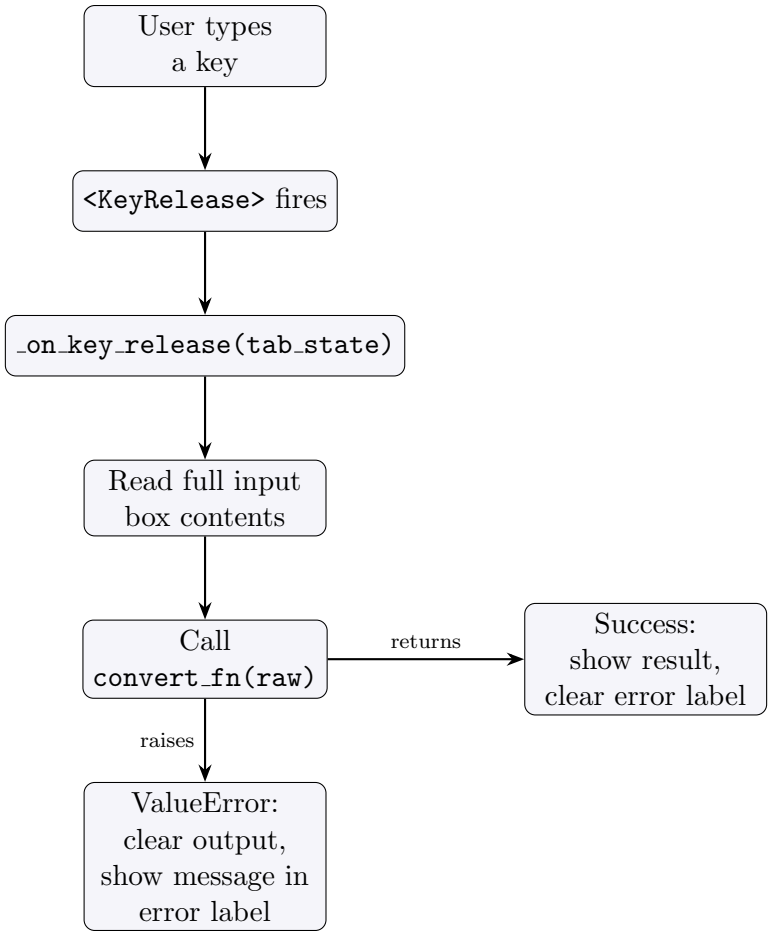
Each tab's state (its input box, output box, error label, and which conversion function it uses) is stored in a small dictionary, `tab_state`, rather than as separate named variables, so the same event-handling code can operate on "whichever tab this is" generically.

3.2 Live conversion on every keystroke

The input `Text` widget of each tab is bound to the `<KeyRelease>` event — a callback that fires every time a key is released while that widget has focus:

```
input_box.bind(
    "<KeyRelease>", lambda event, s=tab_state: self._on_key_release(s)
)
```

`lambda` here defines a small anonymous function used only as this one callback; `s=tab_state` captures the specific tab's state dictionary at binding time, so both tabs' input boxes call the same `_on_key_release` method but each with its own data.



Some implementation details worth noting:

- The output `Text` widget is created with `state="disabled"` so the user cannot type into it directly; `_on_key_release` briefly flips it back to `"normal"` to update its contents, then disables it again.
- If the input box is empty (after stripping whitespace), the handler clears the output and the error label and returns early, instead of trying to convert an empty string.
- Because the callback reruns on *every* keystroke, it also runs on invalid intermediate states — for example, a partially typed Morse sequence. This is expected: the `ValueError` is caught at this layer precisely so a partial or invalid input shows a small inline message instead of stale output or a crash.

3.3 Copy and clear

`_copy_to_clipboard` reads the output box's full text (from position `"1.0"`, meaning line 1 column 0, to `"end-1c"`, the end of the text minus the automatic trailing newline Tkinter always adds) and places it on the system clipboard using Tkinter's built-in `clipboard_clear`/`clipboard_append`, needing no third-party library. `_clear` empties both boxes and the error label for that tab.

4 The playback panel

The playback panel sits below the notebook and is shared by both tabs rather than duplicated per tab: there is one `Canvas` (a Tkinter widget for drawing simple shapes) and one `Play` button.

4.1 Choosing which Morse string to play

Because playback is shared, it needs a rule for which tab's content to animate when the user presses Play. `_current_tab_state` implements that rule:

```
for state in (self.text_to_morse_tab, self.morse_to_text_tab):
    box = (state["output_box"] if state["playback_source"] == "output"
           else state["input_box"])
    content = box.get("1.0", "end-1c").strip()
    if content:
        return content
return ""
```

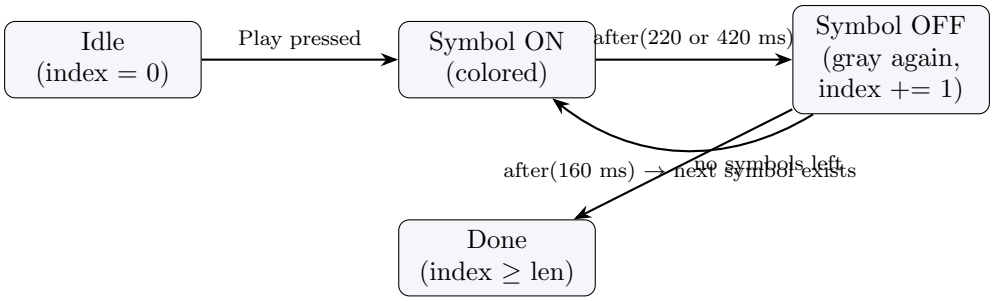
Each tab was tagged at construction time with a `playback_source`: the Text-to-Morse tab is tagged `"output"` (its *output* box already holds Morse code), and the Morse-to-Text tab is tagged `"input"` (its *input* box is the one that holds Morse code, since its output box holds plain text). The function checks the Text-to-Morse tab first; if that tab's Morse output is non-empty, it wins. Only if it is empty does the function fall back to the Morse-to-Text tab's input. This is a priority order, not a "most recently edited" rule — if both tabs happen to have content, the Text-to-Morse tab's output always plays.

4.2 Rendering dots and dashes

`_start_playback` extracts just the dot/dash characters from the chosen string (`[c for c in morse if c in ".-"]` — a *list comprehension*, a compact loop-and-filter, that discards spaces and slashes), then calls `_render_symbols` to draw one shape per symbol on the canvas, left to right: a small oval for each dot, a wider rectangle for each dash, all initially filled with `SYMBOL_OFF` (a muted gray). Each shape's canvas id is stored alongside its symbol in `self._symbol_boxes` so the playback step can look them up in order.

4.3 Timing state machine

Playback advances through the symbols one at a time using `root.after(ms, callback)` — Tkinter's way of scheduling a function to run once, after a delay, without blocking the rest of the program (there is no separate thread; Tkinter's own event loop simply calls the function later). Three constants control timing: `PLAYBACK_DOT_MS = 220` (how long a dot stays lit), `PLAYBACK_DASH_MS = 420` (how long a dash stays lit, roughly double a dot, matching the real Morse convention that a dash is three units long against a one-unit dot), and `PLAYBACK_GAP_MS = 160` (a pause between symbols so consecutive dots/dashes are visibly distinct rather than blurring together).



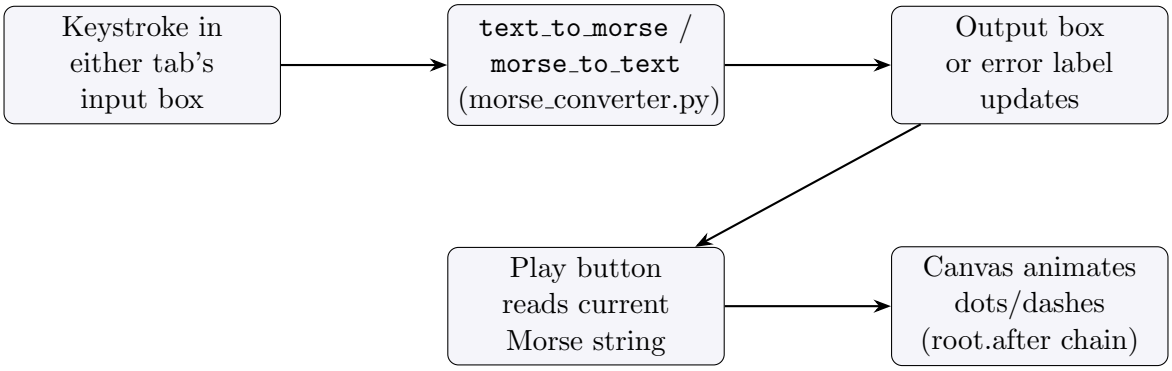
Walking the code behind this diagram, `_advance_playback` is the function that both starts each symbol and is scheduled to run again for the next one:

```
def _advance_playback(self) -> None:
    if self._play_index >= len(self._playback_symbols):
        self._playback_job = None
        return
    box, symbol = self._symbol_boxes[self._play_index]
    color = DOT_ON if symbol == "." else DASH_ON
    duration = PLAYBACK_DOT_MS if symbol == "." else PLAYBACK_DASH_MS
    self.playback_canvas.itemconfigure(box, fill=color)

    def turn_off():
        self.playback_canvas.itemconfigure(box, fill=SYMBOL_OFF)
        self._play_index += 1
        self._playback_job = self.root.after(PLAYBACK_GAP_MS, self._advance_playback)

    self._playback_job = self.root.after(duration, turn_off)
```

This is *recursive scheduling*: `_advance_playback` does not loop internally; instead, each call schedules exactly one future call (`turn_off`, which itself schedules the next `_advance_playback`) via `root.after`, and the chain naturally ends when `_play_index` reaches the end of the symbol list. The base case (`index >= len(symbols)`) sets `self._playback_job = None` and returns without scheduling anything further, which is how the animation stops.

Pressing Play again while an animation is already running cancels the in-flight schedule first (`self.root.after_cancel(self._playback_job)` in `_start_playback`) before starting a fresh one, so repeated presses cannot stack up multiple concurrent animations fighting over the same canvas shapes.

5 Summary of data flow



6 What is intentionally out of scope

Per the README, there is no persistence (nothing is saved to disk), no network calls, and no configuration file. The playback is purely visual; there is no audio, since there is no cross-platform, dependency-free way to play a tone from the Python standard library alone (`winsound` is Windows-only). There are no automated unit tests committed to the repository.